

MITRA v2

Self-Hosted Agentic AutoML — Improved Design Plan

Low-Level Specifications · 8-Person Sprint · 14 Risks Resolved

Field	Value
Version	v2 — improvements over initial design plan
Team	8 people, 2 hrs/day, 14 working days (28 hrs/person)
UI	Streamlit (3 pages) + SSE event bus
Agent framework	Google ADK (adk-python)
Agents	8 agents, 1 per person: DataValidator, MetadataGen, FeatureSelection, ModelSelection, Classification, Regression/USL, Judge, HPT
Compute	Ray (local head + workers)
BYOM	LiteLLM factory → OpenAI / Anthropic / Gemini
Server	FastAPI + Uvicorn (self-hosted, Docker image)
New in v2	Data Validator agent · SSE event bus spec · Agent I/O contracts · Template library LLD · Judge full decision tree · 14-risk matrix · 8-person hour-level sprint plan

This plan extends the initial design with four key improvements: (1) every component has a low-level specification instead of just a label, (2) all inter-agent data contracts are explicitly typed, (3) 14 identified risks are mitigated with concrete code-level fixes, (4) the 8-person sprint is planned at task-hour granularity so every engineer owns at least one agent end-to-end.

Table of Contents

#	Section	Page
1.	What Changed from v1 — Betterments	3
2.	Resolved Ambiguities	4
3.	System Architecture (refined)	5
4.	End-to-End Pipeline with Data Contracts	6
5.	SSE Event Bus — Low-Level Design	9
6.	Agent I/O Contracts (all 8 agents)	11
7.	Low-Level Design: Data Validator Agent	12
8.	Low-Level Design: Metadata Gen Agent	13
9.	Low-Level Design: Feature Selection Agent	14
10.	Low-Level Design: Model Selection Call	15
11.	Low-Level Design: Family Agents (Classification / Regression / USL)	16
12.	Low-Level Design: Template Library	17
13.	Low-Level Design: Judge Agent	19
14.	Low-Level Design: HPT Agent	21
15.	Low-Level Design: Error + Debug Agent	22
16.	Low-Level Design: Ray Wrapper	23
17.	Risk Register v2 — 14 Risks with Code-Level Mitigations	24
18.	8-Person Sprint Plan (2 hrs/day × 14 days)	26
19.	Integration Test Plan	29
20.	Acceptance Criteria v2	30

1. What Changed from v1 — Betterments

The v1 plan identified the right components but left most of them at label-level. Every item below is a concrete improvement with a rationale.

#	Betterment	Why it matters
B 1	Added Data Validator Agent as the very first stage	v1 let bad data silently corrupt encoding. Nulls >80%, zero-variance cols, wrong file format all now fail fast with a UI fix hint.
B 2	Defined the SSE Event Bus as an <code>asyncio.Queue</code> per session with a typed event schema	v1 said 'SSE feed' but gave no design. Without a queue, agents and the HTTP layer would need shared in-process state — a threading hazard.
B 3	Specified inter-agent I/O contracts : exact file names, JSON schemas, and who writes vs reads each	v1 mentioned file names but not schemas. Agents built in parallel will otherwise silently misread each other's outputs.
B 4	Gave the Template Library a concrete structure: Jinja2 <code>train.py</code> , <code>hp_space.yaml</code> , <code>resources.yaml</code> per family	v1 said 'template library' with no file shape. Without Jinja2 placeholders and <code>hp_space.yaml</code> , the HPT agent has no search space to work with.
B 5	Made the Judge Agent decision tree explicit: gap threshold, floor threshold, max-retries, SHAP OOM guard	v1's judge emitted a one-word verdict. It had no defined thresholds, so two engineers would build incompatible implementations.
B 6	USL Agent fully specified : K-Means / DBSCAN / Isolation Forest, cluster-count heuristic, evaluation via silhouette / Davies-Bouldin	v1 named USL Agent but gave zero spec. It was a risk that would surface on Day 10.
B 7	14-risk matrix with code-level mitigations (not just policy statements)	v1 had 10 risks with policy-level mitigations ('add a guardrail'). v2 gives the exact function/tool/config that implements each mitigation.
B 8	8-person sprint at task-hour granularity, 1 agent per person, parallel tracks identified	v1 planned for 3 people. 8 people on the same codebase without track separation will cause merge conflicts and unclear ownership.
B 9	Integration test plan with two canonical fixtures (<code>iris.csv</code> , <code>cats-dogs-10.zip</code>)	v1 listed acceptance criteria but no test fixtures or automated assertions. The demo would rely on manual checks.
B 10	metadata.json strict schema with JSON Schema validation at write-time	v1 showed example JSON but no schema. An LLM writing metadata with a different key name would silently break Feature Selection Agent on Day 9.

2. Resolved Ambiguities

The following questions were left open in v1 or in the meeting transcript. Each is closed with a concrete decision.

Ambiguity	Decision
How does the USL Agent know which columns are features if there is no output column?	metadata.json always includes output_cols : [] for unsupervised problems. The agent uses all non-dropped columns as features. If output_cols is non-empty, USL is never chosen.
Which port does bin/mitra expose to the host?	Streamlit: 8501, FastAPI: 8001, Ray head: 6379, Ray dashboard: 8265. All four are EXPOSE'd in the Dockerfile. Only 8501 and 8001 are mapped by default in docker run examples.
What happens when two models finish training concurrently on Ray?	Each model_NNN/ subdirectory is created atomically before ray.remote() is called. Writers never conflict. The leaderboard is re-sorted on every GET /leaderboard call by reading all metrics.json files.
Does the Error/Debug Agent have a loop limit?	Yes. Max 3 self-correction attempts per script. On the 4th failure, verdict = abort and the model is skipped. The limit is stored in session config, not hard-coded, so it is tunable.
How does the Feature Selection Agent avoid writing features that do not exist?	It outputs a feature_selection.json with keep and drop arrays that are validated against data_encoded.csv column headers before the downstream scaler runs. Mismatch = validation error + abort.
What if the LLM returns model names that have no template?	TemplateResolver returns a fallback flag. The family agent then invokes the write-from-scratch path with the LLM, using the closest template as a few-shot example in the prompt.
How does Streamlit P2 receive SSE events without a full page refresh?	Streamlit does not support native SSE. P2 uses st.empty() + a Python requests.get(..., stream=True) loop in a st.fragment with a 500ms re-run interval. Each event appends to a session_state log list.
What is mini_data.csv exactly?	pandas.describe(include='all') + dtype per column + null_count + unique_count + is_pii_suspect (bool, from column-name heuristic). Capped at 1000 rows for sampling. Always generated before any LLM call.

3. System Architecture (Refined)

The core five-tier structure is preserved from v1. Three refinements: (1) the SSE Event Bus is now a first-class component, not just a note; (2) the Data Validator is the first stage before any LLM touch; (3) the LiteLLM client factory is the *only* path to any LLM — no agent makes a direct HTTP call.

Figure 1 — MITRA v2: Full System Architecture

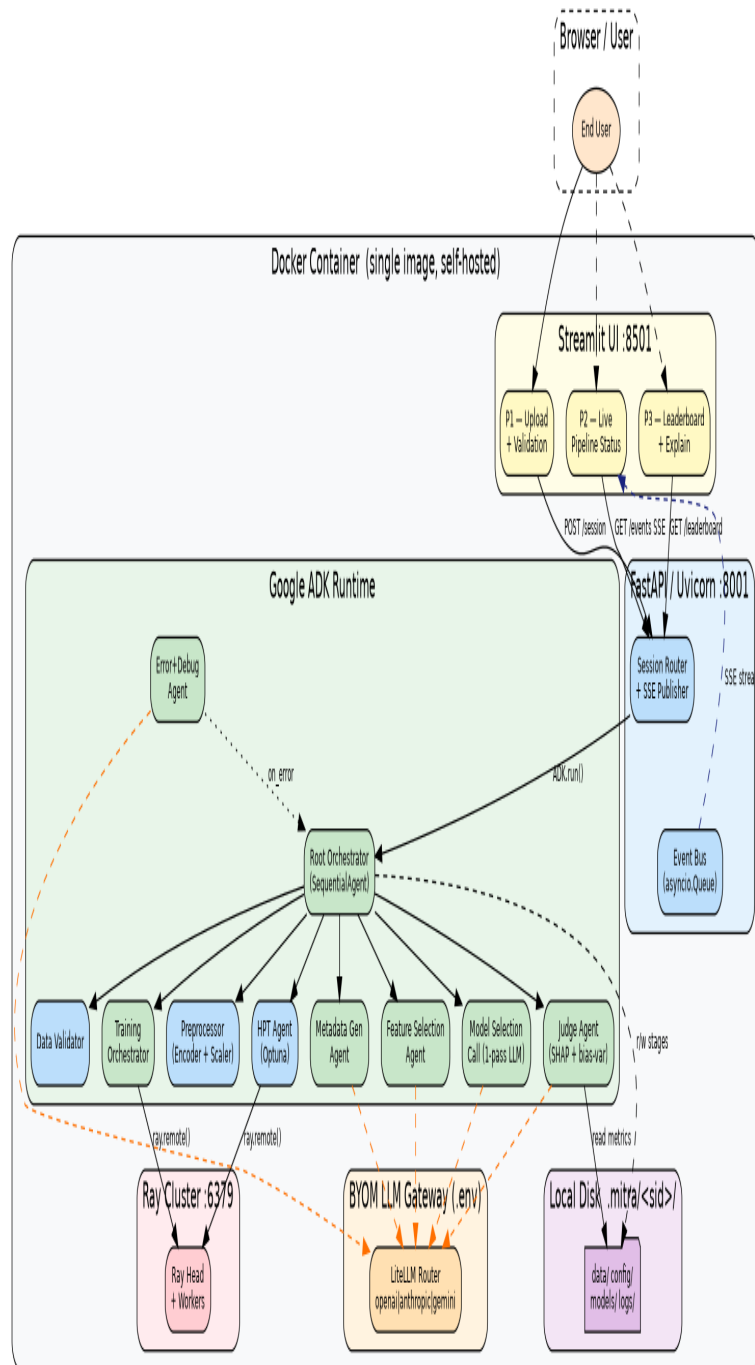


Figure 1 — Refined system architecture. Orange dashed = LLM egress (always via factory). Blue dashed = SSE stream to browser. All components run inside one Docker container.

4. End-to-End Pipeline with Data Contracts

Every arrow in this diagram carries a **named artifact**. No stage reads a file it did not explicitly receive as input. The Data Validator is new — it gates all subsequent work and is the cheapest place to fail.

Figure 2 — End-to-End Pipeline with Inter-Stage Data Contracts

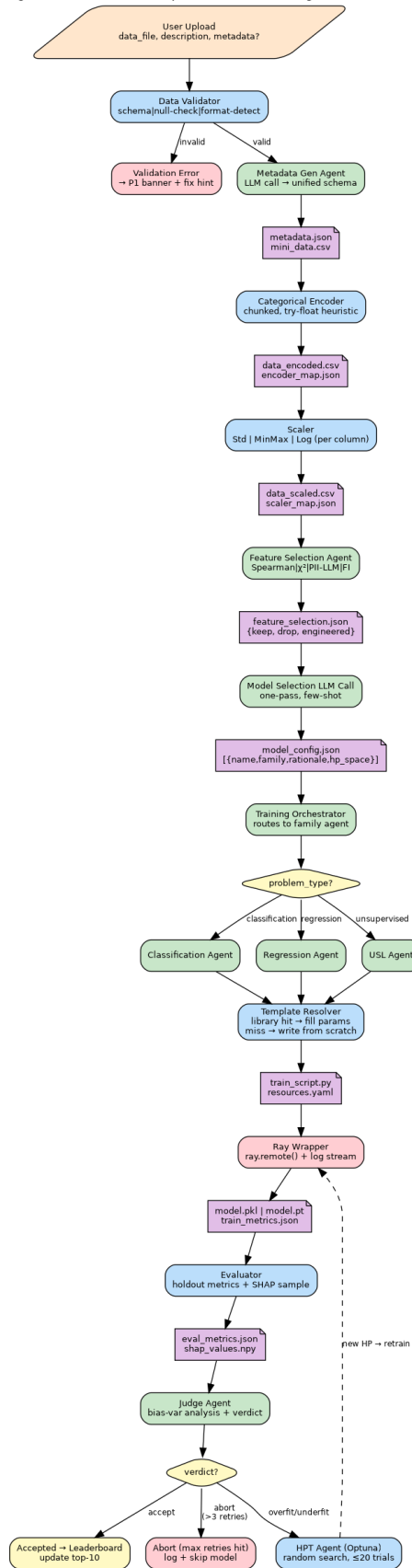


Figure 2 — Full pipeline. Named artifacts on every edge. Green = accepted model path. Pink = error/abort path. Dashed = HPT feedback loop.

4.1 Stage-by-Stage Data Contract Table

Stage	Reads	Writes	Owner	Type
Data Validator	data_file, type_hint	validation_report.json	P3	Python
Metadata Gen	mini_data.csv, description, metadata? (raw)	metadata.json, mini_data.csv	P4	LLM Agent
Encoder	data.csv (chunked), metadata.json	data_encoded.csv, encoder_map.json	P3	Python
Scaler	data_encoded.csv, metadata.json	data_scaled.csv, scaler_map.json	P3	Python
Feature Selection	mini_data.csv, metadata.json, data_encoded header	feature_selection.json	P5	LLM Agent
Model Selection	metadata.json, feature_selection.json, mini_data	model_config.json	P6	LLM Call
Training Orch	model_config.json	(routes to family agent)	P6	LLM Agent
Family Agent	model_config.json[i], templates/	train_script.py, resources.yaml	P6	LLM Agent
Ray Wrapper	train_script.py, data_scaled.csv	model.pkl pt, train_metrics.json	P1	Python
Evaluator	model.pkl pt, data_scaled.csv (holdout)	eval_metrics.json, shap_values.npy	P1	Python
Judge Agent	train+eval metrics, shap_values	verdict.json	P7	LLM Agent
HPT Agent	model_config[i], hp_space.yaml, verdict.json	new_hp.json	P7	Python
Error+Debug	error_log, failed_script	patched_script, error_analysis	P2	LLM Agent

4.2 metadata.json — Full Schema

This schema is the single source of truth. Every agent reads this file. Validated with `jsonschema.validate()` immediately after the Metadata Gen Agent writes it.

```
{
  "$schema": "...",
  "type": "object",
  "required": ["problem_type", "output_cols", "input_cols", "drop_cols", "col_types", "data_format", "row_count", "col_count"],
  "properties": {
    "problem_type": {
      "enum": ["classification", "regression", "unsupervised"]
    },
    "output_cols": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "input_cols": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "drop_cols": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "col_types": {
      "type": "object",
      "additionalProperties": {
        "enum": ["numeric", "categorical", "text", "datetime", "image_path"]
      }
    },
    "data_format": {
      "enum": ["tabular", "image"]
    },
    "row_count": {
      "type": "integer",
      "minimum": 10
    },
    "col_count": {
      "type": "integer",
      "minimum": 1
    },
    "user_description": {
      "type": "string",
      "minLength": 20
    },
    "class_balance": {
      "type": "object"
    } // classification only
  }
}
```


5. SSE Event Bus — Low-Level Design

▲ **Improvement from v1:** v1 said 'SSE feed' with no design. This section gives the complete implementation: queue shape, FastAPI generator, Streamlit consumer, and agent-side tool signature.

Figure 4 — SSE Event Bus: How Agent Progress Reaches the Browser

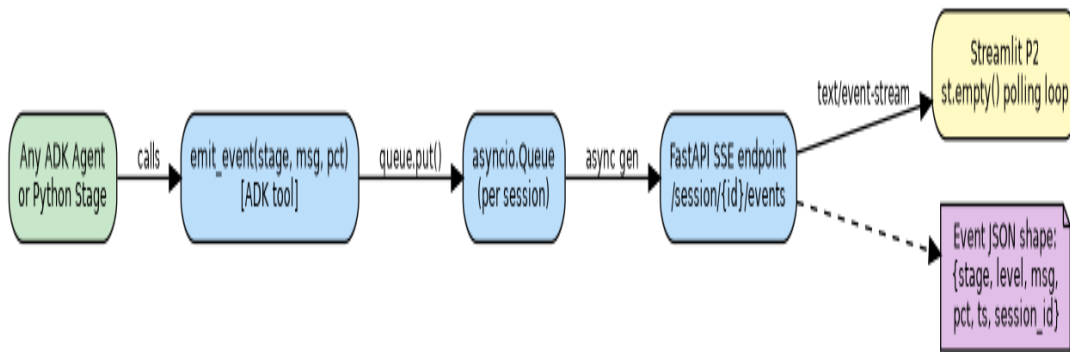


Figure 4 — SSE event flow: agent tool → asyncio queue → FastAPI generator → Streamlit polling loop.

5.1 Event Schema

```
{
  "session_id": "",
  "stage": "feature_selection",
  "level": "info" | "warn" | "error",
  "msg": "chi-square on 12 cols",
  "pct": 42, // 0-100 within this stage
  "ts": "2026-05-24T10:32:11Z"
}
```

5.2 FastAPI SSE Endpoint

```
async def event_stream(session_id: str):
    q = session_queues[session_id] # asyncio.Queue, created at POST /session
    while True:
        event = await asyncio.wait_for(q.get(), timeout=300)
        if event is SENTINEL: break
        yield f"data: {json.dumps(event)}\n\n"

@app.get("/session/{sid}/events")
async def stream(sid: str):
    return StreamingResponse(event_stream(sid),
                             media_type="text/event-stream")
```

5.3 emit_event() — ADK Tool (agent side)

```
def emit_event(stage:str, msg:str, pct:int=0, level:str="info") -> str:
    """ADK FunctionTool that any agent can call to push a status update."""
    event = {"session_id": ctx.session_id, "stage": stage, "level": level,
            "msg": msg, "pct": pct, "ts": utcnow()}
    asyncio.get_event_loop().call_soon_threadsafe(q.put_nowait, event)
    return "ok"
```

■ **Risk addressed:** R4 — SSE queue deadlock: the queue is unbounded (maxsize=0). If Streamlit disconnects, the generator exits and the SENTINEL drains the queue. Ray workers are unaffected.

6. Agent I/O Contracts (all 8 agents)

▲ **Improvement from v1:** v1 mentioned file names. v2 specifies exact keys, types, and validation for every agent boundary.

Figure 3 — Agent I/O Contracts (files passed between stages)

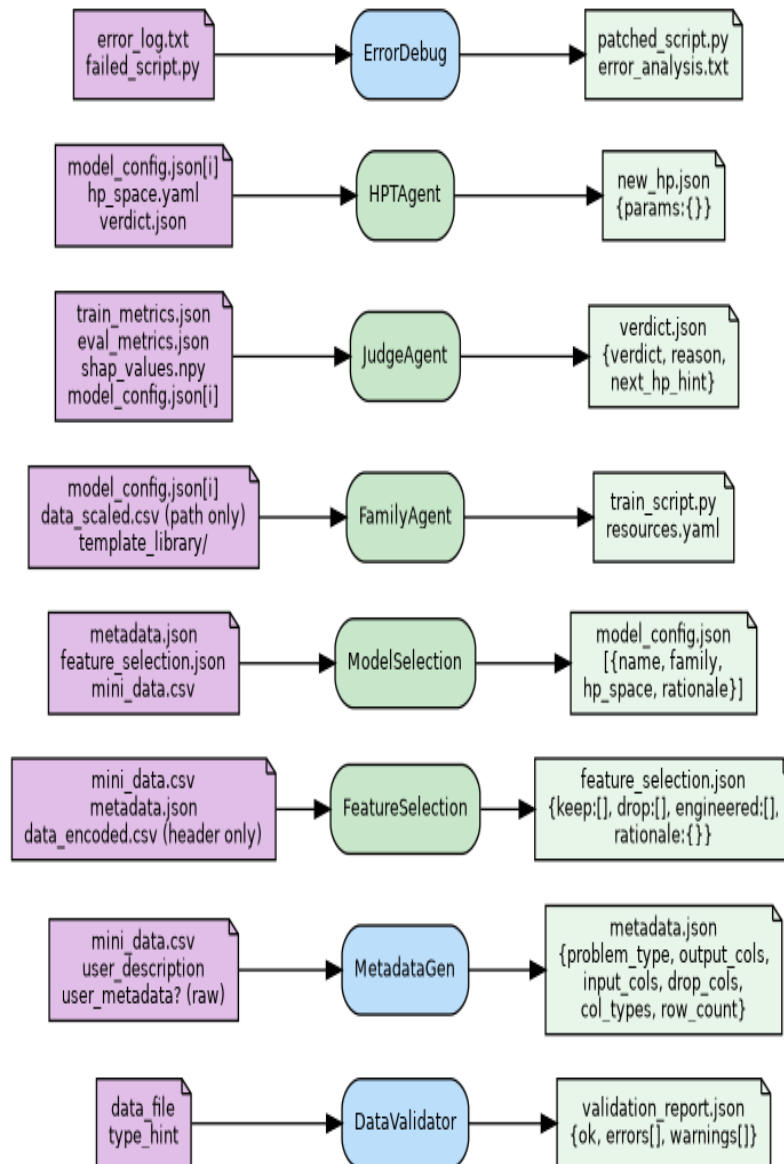


Figure 3 — Agent I/O contract map. Left = inputs (purple), right = outputs (green). Every boundary is a named file validated by the receiving agent.

7. Low-Level Design: Data Validator Agent

Owner	Type	Tools
P3	Python (deterministic, no LLM)	read_file, write_file

Inputs

- data_file (path)
- type_hint: tabular|image

Outputs

- validation_report.json → {ok:bool, errors:[], warnings:[], detected_format:str, row_count:int, col_count:int}
- On ok=false: error surfaced to P1 UI. Pipeline halts.

Key guardrails

- Never loads the full file into RAM. Uses chunked pandas read (chunksize=10000).
- Error codes are structured enums: NULL_EXCESS | ZERO_VARIANCE | BAD_FORMAT | ROW_COUNT_TOO_LOW | IMAGE_LAYOUT_INVALID.
- Image mode: walks ZIP, checks every subfolder for at least 5 images; rejects ZIP with flat structure.
- Tabular mode: rejects if any column has >80% nulls, or if row_count < 50.

Prompt skeleton (agents/data_validator_agent/prompt.md)

```
# Not an LLM agent – pure Python. # File: mitra/agents/data_validator/validator.py def
validate(data_path, type_hint) -> ValidationReport: if type_hint == 'image': return
validate_image_zip(data_path) return validate_tabular(data_path) # chunked read
```

■ Risk addressed: R1 variant — validator itself must not load the full file. Uses pd.read_csv(chunksize=10000) and streams column stats. Never calls pd.read_csv() without chunksize.

8. Low-Level Design: Metadata Gen Agent

Owner	Type	Tools
P4	LLM Agent (single LLM call + schema validation)	read_file, write_file

Inputs

- mini_data.csv (always present)
- user_description (string, ≥20 words)
- user_metadata.json (optional raw file from user)

Outputs

- metadata.json — validated against §4.2 JSON Schema
- If schema validation fails: one retry with the validation error appended to prompt. If 2nd attempt also fails: abort with structured error.

Key guardrails

- Output validated with jsonschema.validate() immediately after LLM returns.
- User metadata treated as a hint — agent merges it with mini_data inferences; user cannot override row_count or col_count.
- Minimum user_description length enforced at P1 UI level (20 words) AND here (checked before LLM call).
- No tool access to data.csv — only mini_data.csv is in the tool context.

Prompt skeleton (agents/metadata_gen_agent/prompt.md)

```
You are a data scientist analyzing a dataset to produce metadata.json. ## Input you have
mini_data.csv: {mini_data_content} User description: {user_description} User-supplied
metadata (optional, treat as hint): {user_metadata} ## Task Produce ONLY a valid JSON
object matching this schema: {schema} Rules: - Never hallucinate column names. Only use
columns from mini_data. - If user_description mentions a target column, set it in
output_cols. - Drop PII columns (phone, aadhaar, SSN, email, id) into drop_cols. - Output
ONLY the JSON. No prose, no markdown fences.
```

9. Low-Level Design: Feature Selection Agent

Owner	Type	Tools
P5	LLM Agent (tools + stats)	read_file, write_file, python_exec

Inputs

- mini_data.csv
- metadata.json
- data_encoded.csv (header row only — not data)

Outputs

- feature_selection.json: {keep:[], drop:[], engineered:[{name, formula}], rationale:{col:reason}}
- Validated: keep $\cup$ drop == all non-drop_cols from metadata.json (no orphan columns).

Key guardrails

- **HARD GUARDRAIL:** python_exec tool has data_encoded.csv path REMOVED from its working context. Only the header row is provided as a string.
- Agent must not call pd.read_csv on the full file. If it tries, the tool returns PermissionError.
- Spearman/Pearson/ χ^2 are computed by a Python helper on mini_data — the agent calls the helper, not raw pandas.
- engineered features limited to degree-2 polynomial and ratio features. No arbitrary lambda.
- Output validated: every column in keep must exist in data_encoded.csv header.

Prompt skeleton (agents/feature_selection_agent/prompt.md)

You are a feature engineering expert. ## What you have access to mini_data.csv (statistics summary, NOT the raw data) metadata.json (problem type, output cols, drop cols) data_encoded.csv HEADER: {header_row} ## Correlation results (pre-computed) {spearman_table} {chisq_table} {feature_importance_table} ## Task Output ONLY feature_selection.json. Rules: - NEVER read or reference the full data file. - Drop PII cols, low-importance cols (FI < 0.01), and user-specified drop_cols. - Suggest up to 3 engineered features if correlation gain is plausible. - Every column you keep must appear in the header above.

10. Low-Level Design: Model Selection Call

Owner	Type	Tools
P6	Single LLM call (no tools, no loop)	— (none)

Inputs

- metadata.json
- feature_selection.json
- mini_data.csv

Outputs

- model_config.json: [{name, family, rationale, hp_space:{param:range}, priority}] — sorted by expected fit

Key guardrails

- Single pass — no tools, no agent loop. Response parsed immediately; if JSON invalid: one retry.
- Output list length capped at 8 models. Prompt instructs: 'return between 3 and 8 models'.
- family must be one of: xgboost | random_forest | logistic_reg | svm | mlp | cnn_image | kmeans | dbscan | isolation_forest.
- hp_space values are ranges/lists, not single values — they feed directly into Optuna's suggest_* API.

Prompt skeleton (agents/model_selection_call/prompt.md)

You are an AutoML expert. Given the dataset profile below, select 3-8 ML models. Dataset profile: {metadata_json} Selected features: {feature_selection_json} Statistical summary: {mini_data_excerpt} ## Output format (JSON only, no prose)

```
[{"name":"xgboost_v1","family":"xgboost","priority":1,"rationale":"Tabular classification, mixed types",
"hp_space":{"n_estimators":[50,500],"max_depth":[3,10],"lr":[0.01,0.3]}}, ...] ## Rules - family must be one of:
xgboost|random_forest|logistic_reg|svm|mlp|cnn_image|kmeans|dbscan|isolation_forest - 3-8 models only. Sort by expected performance (best first). - Output ONLY the JSON array.
```

11. Low-Level Design: Family Agents (Classification / Regression / USL)

▲ **Improvement from v1:** USL Agent was completely unspecified in v1. All three family agents now have algorithm sets, evaluation metrics, and prompt skeletons.

Agent	Algorithms	Evaluation Metrics	Key constraints
Classification	XGBoost, Random Forest, Logistic Reg, SVM, MLP, CNN (images)	Accuracy, F1-macro, ROC-AUC, Confusion Matrix	Class imbalance: stratified split; if imbalance >5:1, force class_weight='balanced'
Regression	XGBoost-Reg, Random Forest-Reg, Ridge, Lasso, MLP-Reg	RMSE, MAE, R², residual plot	Outlier rows (z >3) flagged in eval report but NOT auto-dropped
USL Agent	K-Means (k=2.. $\sqrt{n}$), DBSCAN (eps from nearest-neighbor), Isolation Forest	Silhouette, Davies-Bouldin, Calinski-Harabasz; Isolation Forest → anomaly_ratio	output_cols must be empty in metadata.json. If not empty, orchestrator rejects USL routing.

11.1 Template Resolution Protocol (all family agents)

- Step 1: Read model_config.json[i].family.
- Step 2: Look up mitra/templates/{family}/train.py.jinja2. If exists → go to Step 3. Else → Step 5.
- Step 3: Read hp_space.yaml from same folder. Merge with model_config[i].hp_space (config overrides template defaults).
- Step 4: Render Jinja2 template with {n_classes, input_dim, output_cols, hp_dict}. Write train_script.py. Done.
- Step 5 (fallback): Call LLM with the closest template as a few-shot example. LLM writes the full script. Run syntax check (py_compile). If fails → retry once.

■ **Risk addressed:** R3 — Template regen: Step 2 means the LLM is only called for genuinely novel architectures. All standard models (XGBoost, RF, MLP, CNN) will always hit Step 4.

12. Low-Level Design: Template Library

▲ **Improvement from v1:** v1 said 'template library' with no file structure. v2 defines every file in every template folder.

Figure 5 — Template Library Structure & Resolution Flow

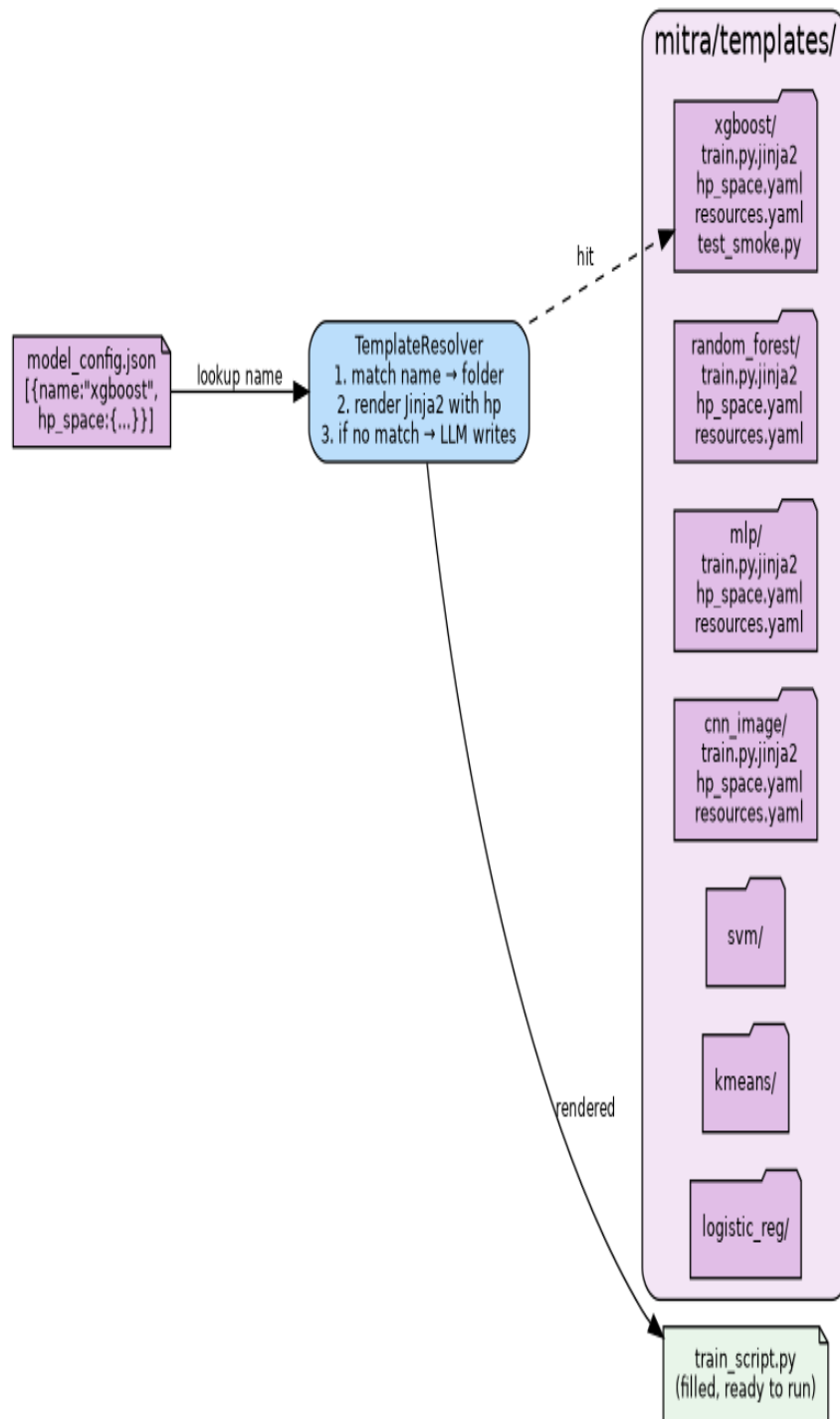


Figure 5 — Template library structure. Each folder ships four files. TemplateResolver does a name→folder lookup, then Jinja2 render.

12.1 Template Folder Anatomy

```
mitra/templates/xgboost/  
train.py.jinja2 # Jinja2 with {{n_estimators}}, {{max_depth}}, {{lr}}, {{output_cols}}  
hp_space.yaml # Optuna search space defaults  
resources.yaml # {cpu: 2, gpu: 0, mem_gb: 4}  
test_smoke.py # pytest fixture: trains on 50 rows of iris, asserts accuracy > 0.5
```

12.2 hp_space.yaml format

```
# xgboost/hp_space.yaml n_estimators: {type: int, low: 50, high: 500, step: 50} max_depth:  
{type: int, low: 3, high: 10} learning_rate: {type: float, low: 0.01, high: 0.3, log:  
true} subsample: {type: float, low: 0.6, high: 1.0} colsample: {type: float, low: 0.6,  
high: 1.0}
```

The HPT Agent reads this file and calls `trial.suggest_int()`, `trial.suggest_float()` accordingly. No hard-coded search spaces in agent prompts.

13. Low-Level Design: Judge Agent

▲ **Improvement from v1:** v1 gave a one-line description. v2 gives the full decision tree with configurable thresholds, SHAP OOM guard, and a typed verdict schema.

Figure 7 — Judge Agent: Full Decision Logic

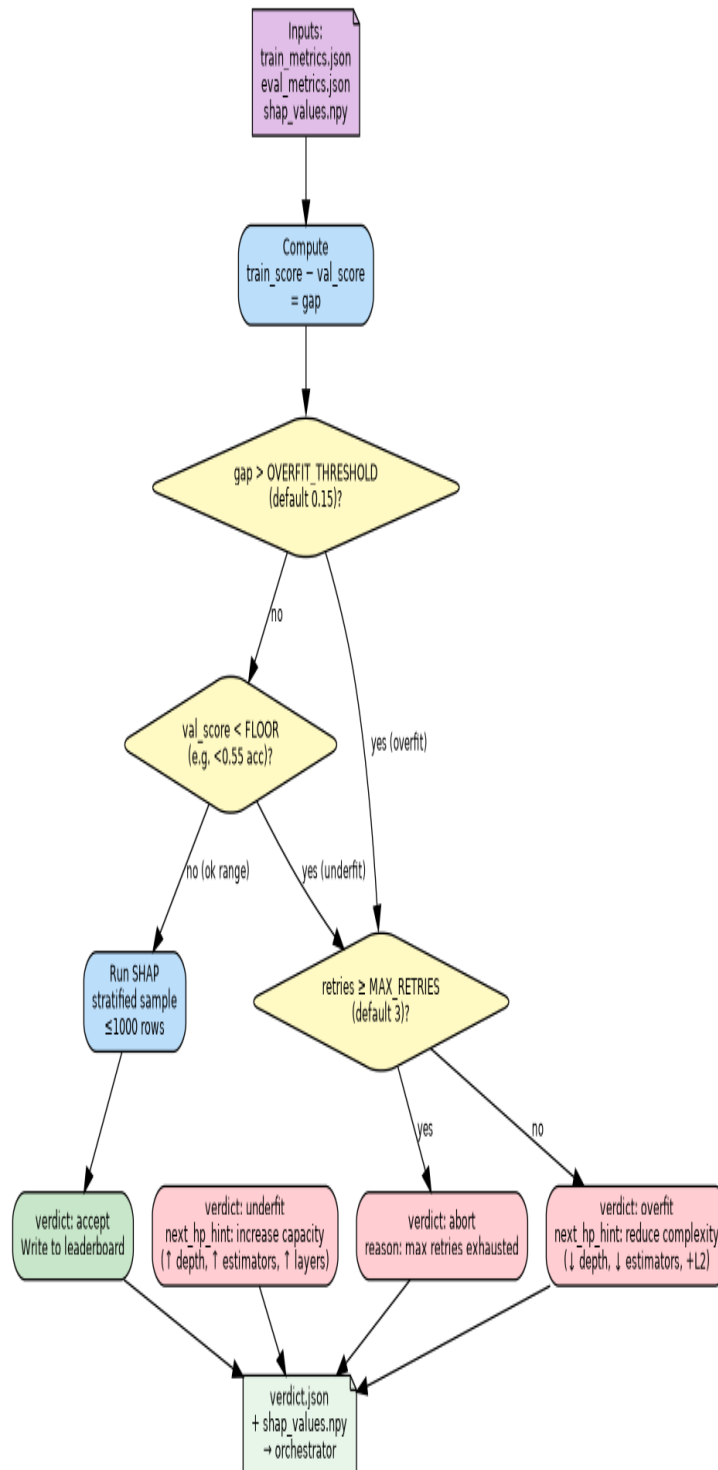


Figure 7 — Judge Agent full decision tree. Thresholds are config-driven, not hard-coded.

13.1 Configurable Thresholds (session config)

```
OVERFIT_THRESHOLD: 0.15 # train_score - val_score > this → overfit FLOOR_SCORE: 0.55 #  
val_score < this (classification) → underfit FLOOR_R2: 0.10 # R² < this (regression) →  
underfit MAX_RETRIES: 3 # per model before abort SHAP_MAX_ROWS: 1000 # cap to avoid OOM on  
large models SHAP_TIMEOUT_SEC: 120 # kill SHAP computation if it exceeds this
```

■ **Risk addressed:** R14 — SHAP OOM: SHAP is run in a Ray remote task with memory limit (resources.yaml). If it times out (120s), the verdict is still computed using metrics only; SHAP is marked unavailable.

13.2 verdict.json Schema

```
{"verdict": "accept"|"overfit"|"underfit"|"abort",  
  "gap": 0.18,  
  "val_score": 0.81,  
  "retries_used": 1,  
  "reason": "train_acc=0.97 val_acc=0.79, gap=0.18 > threshold 0.15",  
  "next_hp_hint": {"reduce": ["max_depth", "n_estimators"],  
    "increase": ["subsample"]},  
  "shap_available": true}
```

14. Low-Level Design: HPT Agent

Owner	Type	Tools
P7	Python (Optuna, no LLM in loop)	optuna_run, ray_submit

Inputs

- model_config.json[i]
- hp_space.yaml (from template folder)
- verdict.json (with next_hp_hint)

Outputs

- new_hp.json: {trial_id, params:{}, expected_improvement}
- hp_history.json: all trial results appended (for P3 visualization)

Key guardrails

- No LLM in the loop — Optuna runs pure Python. LLM cost per trial would be prohibitive.
- next_hp_hint from Judge Agent biases the Optuna sampler: suggested 'reduce' params are capped at their current value, 'increase' params floored.
- Hard budget: max 20 trials OR 5 minutes wall-clock per model (whichever comes first). Enforced via Optuna callbacks.
- Stratified sampling for class-imbalanced datasets (split preserves class ratios).
- Each trial runs as ray.remote() so the HPT agent process is never blocked.

Prompt skeleton (agents/hpt_agent/prompt.md)

```
# Not a prompt – Python implementation: def objective(trial): hp =
sample_from_space(trial, hp_space, hint=verdict.next_hp_hint) ref =
ray_submit(train_script, hp, data_path) metrics = ray.get(ref) return metrics['val_score']
study = optuna.create_study(direction='maximize',
sampler=optuna.samplers.TPESampler(seed=42)) study.optimize(objective, n_trials=20,
timeout=300, callbacks=[MaxRetriesCallback(3)])
```

15. Low-Level Design: Error + Debug Agent

Owner	Type	Tools
P2	LLM Agent	read_file, write_file, bash, python_exec (restricted)

Inputs

- error_log.txt (last 200 lines)
- failed_script.py
- resources.yaml

Outputs

- patched_script.py
- error_analysis.txt: {error_type, root_cause, fix_applied, confidence}

Key guardrails

- May ONLY write files under .mitra/<session-id>/. MITRA source tree is mounted read-only.
- Max 3 self-correction attempts per failure. On 4th failure: emit abort event, skip model.
- bash tool restricted to: python, pip install (whitelist only), cat, head. No curl, no wget, no git.
- pip install whitelist: scikit-learn, xgboost, lightgbm, torch, shap, optuna, pandas, numpy.

Prompt skeleton (agents/error+_debug_agent/prompt.md)

You are a Python debugging expert. A training script failed. Error log (last 200 lines): {error_log} Failed script: {failed_script} ## Task 1. Identify the root cause (one sentence). 2. Produce a corrected script. Minimal changes only. 3. Output ONLY the corrected script. No explanations. ## Constraints - Do not import packages outside: {whitelist} - Do not read any file outside: .mitra/{session_id}/ - Do not modify any file in: mitra/ (platform source)

16. Low-Level Design: Ray Wrapper

The Ray Wrapper is a **deterministic Python module**, not an agent. It has four responsibilities, each with a clear failure mode.

- **Start:** `ray.init(address='auto', ignore_reinit_error=True)`. If Ray is not running, spawns a local head with `ray.init(num_cpus=os.cpu_count()-1)`. Reports to `/health` endpoint.
- **Submit:** `@ray.remote(num_cpus=N, num_gpus=G)` decorated `run()` function. N and G come from `resources.yaml`. Falls back to CPU-only if no GPU found.
- **Stream logs:** Ray's `ray.get()` is blocking; logs are tailed via a background thread that reads the Ray worker's stdout and emits SSE events every 2 seconds.
- **Teardown:** On DELETE `/session`, `ray.cancel(ref, force=True)` is called for any active refs tied to that session. Ray cluster stays alive for the next session.

■ **Risk addressed:** R5 — Ray fails to start: `/health` checks `ray_ok` on startup. P1 blocks session creation if `ray_ok=false`, showing 'Ray not ready — check Docker resource limits'.

17. Risk Register v2 — 14 Risks with Code-Level Mitigations

Figure 9 — Risk Matrix (14 identified risks)

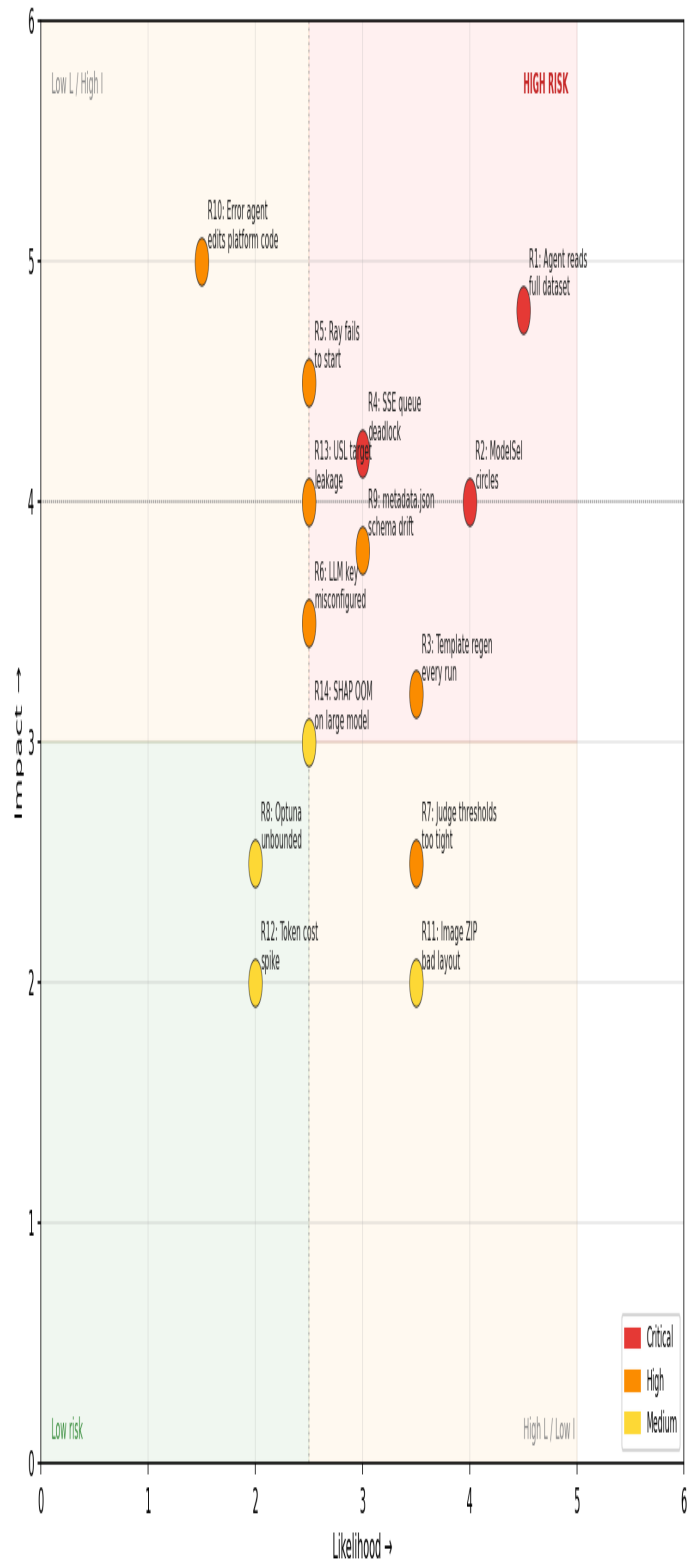


Figure 9 — Risk matrix. X=likelihood, Y=impact. Red=Critical, Orange=High, Yellow=Medium.

17.1 Risk Table

ID	Risk	Sev	Code-Level Mitigation
R1	Agent reads full dataset	Crit	TemplateResolver and all agent tool contexts receive ONLY mini_data.csv path. data_scaled.csv path is injected only into the Ray worker subprocess, never into an ADK context.
R2	ModelSel circles (agent loop)	Crit	Model Selection is a FunctionCall, not an Agent. ADK runner invokes it as a single turn. No tools attached → no loop possible.
R3	Template regen every session	High	TemplateResolver.resolve(family) checks templates/ before any LLM call. Cache miss rate is expected ~0% for standard families.
R4	SSE asyncio.Queue deadlock	High	Queue is unbounded (maxsize=0). FastAPI SSE generator has a 300s timeout. On client disconnect, the generator exits and SENTINEL is pushed to drain pending events.
R5	Ray cluster fails to start	High	/health endpoint polls ray.is_initialized() every 5s. P1 blocks session start with a banner if ray_ok=false. Docker run sets --shm-size=2g to prevent Ray shared-memory OOM.
R6	LLM key misconfigured	High	Startup smoke test: client.complete([{"role": "user", "content": "Reply OK"}]). Fails with exit code 1 and prints the upstream HTTP error. Container does not serve requests.
R7	Judge thresholds too tight	High	OVERFIT_THRESHOLD and FLOOR_SCORE are in session_config.yaml, not hard-coded. Defaults are conservative (0.15, 0.55). User can override via ?overfit_thresh=0.2 query param.
R8	Optuna runs unbounded	Med	study.optimize(n_trials=20, timeout=300). MaxRetriesCallback halts after 3 failed trials. Budget is logged as SSE event.
R9	metadata.json schema drift	High	jsonschema.validate(metadata, SCHEMA) immediately after MetadataGen writes the file. Pipeline halts with a structured error if schema mismatch. Schema version is embedded in the JSON.
R10	Error agent edits platform code	High	MITRA source mounted as read-only Docker volume. bash tool uses a whitelist of allowed paths: only .mitra/. os.path.abspath check before every file write.
R11	Image ZIP bad layout	Med	DataValidator walks the ZIP tree. If any image file is in the root (not in a subfolder), returns IMAGE_LAYOUT_INVALID with a one-line fix instruction shown in P1.
R12	Token cost spike	Med	Per-session token counter. Agents report token usage via emit_event. At 80% of TOKEN_BUDGET (default 100K), a P2 warning banner appears. At 100%, pipeline pauses and asks user.
R13	USL target leakage	High	Training Orchestrator checks: if problem_type=='unsupervised' AND output_cols!=[], reject with USL_TARGET_LEAKAGE error before any model runs.
R14	SHAP OOM on large model	Med	SHAP runs in ray.remote(memory=2*1024**3). If it exceeds 2GB or 120s, Ray kills the task and verdict is computed metrics-only. shap_available=false in verdict.json.

18. 8-Person Sprint Plan (2 hrs/day × 14 days)

Each person owns **one agent end-to-end**: prompt, schema, tests, and integration. P1 (Infra) and P2 (API+SSE) are the platform backbone; their first 4 days must complete before any agent can emit events. All agent owners start with their JSON schema and unit tests on Day 1, even before the infra is live.

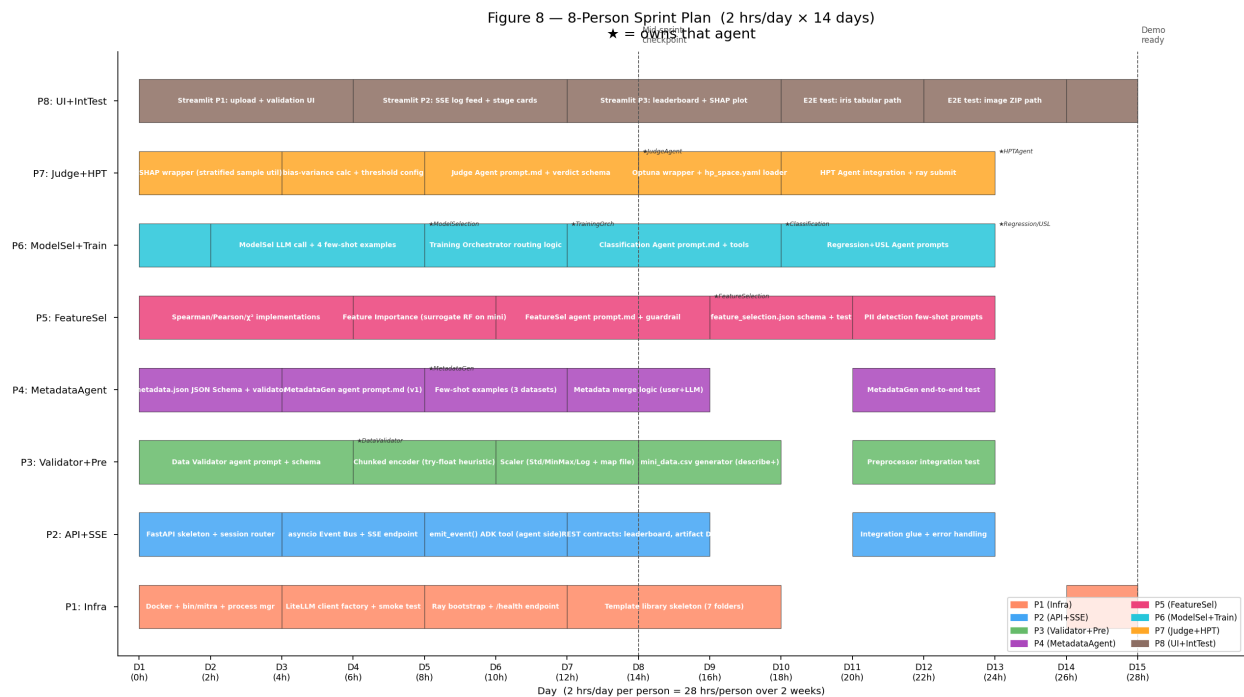


Figure 8 — 8-person sprint. ★ marks agent ownership. Each person = 2 hrs/day × 14 days = 28 hrs total. Mid-sprint checkpoint D8: SSE bus alive, one model trained end-to-end.

18.1 Agent Ownership Matrix

Person	Primary Agent	Secondary tasks	Mid-sprint deliverable (D8)
P1 — Infra	—	Docker, bin/mitra, Ray, template library skeleton	Ray starts cleanly; /health green; 7 template folders created
P2 — API+SSE	—	FastAPI router, asyncio Event Bus, SSE endpoint	POST /session creates queue; GET /events streams events from stub agent
P3 — Validator+Pre	DataValidator	Chunked encoder, scaler, mini_data generator	DataValidator passes iris.csv; data_scaled.csv produced
P4 — Metadata	MetadataGen Agent	JSON Schema validator, merge logic	iris.csv + 30-word description → valid metadata.json
P5 — Features	FeatureSelection Agent	Spearman/ χ^2 helpers, PII few-shot prompts	feature_selection.json produced from iris metadata
P6 — ModelSel+Train	ModelSelection + Classification	Regression+USL prompts, training orchestrator routing	One XGBoost model trained on Ray; metrics.json written
P7 — Judge+HPT	Judge Agent + HPT Agent	SHAP wrapper, Optuna wrapper, hp_space loader	Judge produces verdict.json for the D8 XGBoost model
P8 — UI+IntTest	—	Streamlit P1/P2/P3, E2E test fixtures	P1 upload works; P2 shows SSE events from stub

18.2 Day-by-Day Milestones

Day	Hours spent (8 people)	Milestone	Acceptance
D2	16h	Infra alive	bin/mitra boots; /health green; SSE queue created per session
D4	16h	P1 UI + Validator	Upload CSV in P1; DataValidator returns validation_report.json
D6	16h	Metadata + Preprocessing	mini_data.csv + metadata.json + data_scaled.csv all written for iris.csv
D8	16h	Mid-sprint: one model trained	FeatureSel → ModelSel → XGBoost train on Ray → metrics.json
D10	16h	Judge + HPT loop	Judge produces verdict → HPT runs 5 trials → retrain on Ray
D12	16h	All agents integrated	Full pipeline: iris.csv → leaderboard (3+ models)
D13	16h	P3 UI + SHAP plots	Leaderboard renders; SHAP bar chart for top model in P3
D14	16h	Image path + E2E tests	cats-dogs-10.zip → CNN model → leaderboard; both pytest suites pass

19. Integration Test Plan

▲ **Improvement from v1:** v1 had only acceptance criteria, no test fixtures or automated assertions. v2 ships two canonical fixtures and a pytest suite owned by P8.

19.1 Fixture 1 — iris.csv (tabular classification)

- 150 rows, 4 features, 3 classes — deliberately small so CI completes in <5 minutes.
- Expected: metadata.json sets problem_type=classification, output_cols=[species].
- Expected: feature_selection.json keeps all 4 features (all are informative).
- Expected: model_config.json includes xgboost and random_forest.
- Expected: leaderboard top-1 accuracy ≥ 0.90 .
- Expected: verdict.json verdict=accept (no overfitting on this clean dataset).

19.2 Fixture 2 — cats-dogs-10.zip (image classification)

- 10 images per class (cats/, dogs/) — tiny enough to train quickly without a GPU.
- Expected: DataValidator accepts ZIP (2 subfolders, 10+ images each).
- Expected: model_config.json includes cnn_image.
- Expected: CNN trains without error; accuracy ≥ 0.60 (small dataset, so threshold is lenient).
- Expected: shap_available=false (CNN SHAP is slow; falls back gracefully).

19.3 Negative Test Cases (P8 owns)

Test	Input	Expected behaviour
T-NEG-1	CSV with 80% nulls in column A	DataValidator: NULL_EXCESS error; P1 shows fix hint
T-NEG-2	ZIP with flat structure (no subfolders)	DataValidator: IMAGE_LAYOUT_INVALID; clear message
T-NEG-3	description = 'hello' (2 words)	P1 rejects before POST /session (client-side validation)
T-NEG-4	TYPE=openai but wrong API key	Startup smoke test fails; container exits code 1; no requests served
T-NEG-5	USL dataset with output_cols=['target'] in metadata	Training Orchestrator: USL_TARGET_LEAKAGE error; model skipped
T-NEG-6	LLM returns model_config with family='mystery_net'	TemplateResolver: fallback to write-from-scratch; LLM writes script; smoke test runs

20. Acceptance Criteria v2

Criteria are grouped by tier. All must pass before the Docker image is tagged as stable.

Tier	Criterion	Measured by
Functional	iris.csv → 3+ trained models, top-1 acc $\geq$ 0.90	pytest test_iris_e2e.py
Functional	cats-dogs-10.zip → CNN, acc $\geq$ 0.60	pytest test_image_e2e.py
Robustness	Empty metadata.json + 30-word description → pipeline completes	pytest test_no_metadata.py
BYOM	TYPE=openai → TYPE=anthropic in .env → pipeline completes unchanged	Manual / pytest parametrize
BYOM	TYPE=openai with wrong key → container exits code 1 in < 5s	pytest test_smoke_fail.py
Self-hosting	docker run mitra:latest on clean Ubuntu 22.04 VM → P1 loads	Manual
Cost	iris.csv end-to-end consumes < 50K tokens	Token counter in SSE logs
SSE	P2 progress bars update without page refresh	Playwright E2E test
Reliability	Kill LLM endpoint mid-run → P2 shows error banner, no orphaned Ray jobs	pytest test_llm_failure.py + ray.available_resources() check
Perf	iris.csv full pipeline (train 3 models) completes in < 4 minutes on a 4-core laptop	pytest + time assertion
Security	Error/Debug agent cannot write to mitra/ source tree	pytest test_agent_sandbox.py: attempt write → PermissionError asserted
Data	Feature selection output validated against data_encoded.csv header — no orphan columns	pytest test_feature_schema.py

— End of MITRA v2 Design Plan —